



目录

- ▶ 语言模型概述
- ▶ N-gram语言模型
 - ▶ 模型参数
 - ▶ N-gram模型
 - ▶ 参数估计
 - ▶ 数据平滑
- ▶ 模型应用
- ▶ 语言模型评估



语言模型概述

▶ 基本概念

- ▶ N-gram语言模型又称统计语言模型(Statistic language models, SLM)
- ▶ 利用概率统计理论研究语言的模型;
- ▶ 区别于现在流行的神经语言模型

▶ 语言模型的基础

- ▶ 大规模语料库(data)
 - ▶ 为自然语言统计处理方法的实现提供了可能
- ▶ 统计方法(method)
 - ▶ E.g. 最大似然估计



语言模型概述

▶ 语言建模

- ▶ 任何语言片断都有存在的可能，只是可能性大小不同

$x1=(\text{熊猫}, \text{喜欢}, \text{吃}, \text{竹子})$

$x2=(\text{竹子}, \text{熊猫}, \text{吃}, \text{喜欢})$

$P(x1), P(x2)$, 哪个概率更大?

- ▶ 语言模型：计算一个句子或者一个词序列的概率的模型；或者在已出现的单词序列条件下，预测下一个出现的单词的概率的模型。

$P(x) = ?$



语言模型概述

▶ 模型求解

- ▶ 对于一个文档片段 $x=w_1, w_2, \dots, w_n$ ，语言模型是指对概率 $P(x)=P(w_1, w_2, \dots, w_n)$ 的求解

- ▶ 其联合概率为：

- ▶ $P(A, B)=P(A)P(B|A)$

- ▶ 如果有更多变量的话，根据链式法则：

- ▶ $P(A, B, C, D) = P(A)P(B|A)P(C|A, B)P(D|A, B, C)$

$$P(w_1 w_2 \dots w_n)=?$$



语言模型概述

► 模型求解

► 根据链式法则

$$\begin{aligned} & P(w_1 w_2 \dots w_n) \\ &= P(w_1) P(w_2 | w_1) P(w_3 | w_1 w_2) \dots P(w_n | w_1 w_2 \dots w_{n-1}) \\ &= P(w_1) \prod_{i=2}^n P(w_i | w_1 w_2 \dots w_{i-1}) \end{aligned}$$

- w_i 可以是字、词、短语或词类等等，称为统计基元。通常以“词”代之。
- w_i 的出现跟它前面出现的词有关，称为 w_i 的历史



语言模型概述

► 模型求解

► 语言片段的计算方法

$P(\text{“熊猫, 喜欢, 吃, 竹子”}) =$
 $P(\text{熊猫}) \times P(\text{喜欢} | \text{熊猫}) \times P(\text{吃} | \text{熊猫, 喜欢}) \times P(\text{竹子} |$
 $\text{熊猫, 喜欢, 吃})$

$$P(w_1 w_2 \dots w_n) = P(w_1) \prod_{i=2}^n P(w_i | w_1 w_2 \dots w_{i-1})$$



语言模型概述

▶ 语言模型应用

▶ 机器翻译

$P(\text{high winds tonite}) > P(\text{large winds tonite})$

▶ 拼写检查(纠错)

The office is about fifteen minuets from my house

$P(\text{about fifteen minutes from}) > P(\text{about fifteen minuets from})$

▶ 语音识别

▶ $P(\text{I saw a van}) \gg P(\text{eyes awe of an})$

...



目录

- ▶ 语言模型概述
- ▶ N-gram语言模型
 - ▶ 模型参数
 - ▶ N-gram模型
 - ▶ 参数估计
 - ▶ 数据平滑
- ▶ 模型应用
- ▶ 语言模型评估



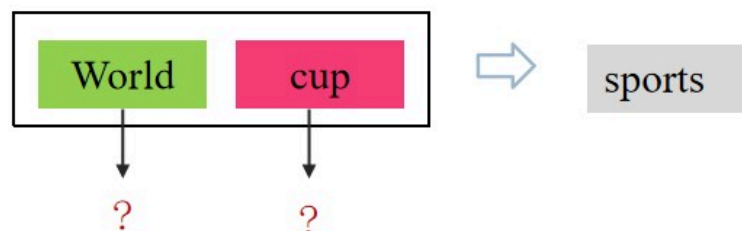
模型参数

▶ 模型复杂度(参数量)

▶ 模型的参数

$$P(w_1 w_2 \dots w_n) = P(w_1) \prod_{i=2}^n P(w_i | w_1 w_2 \dots w_{i-1})$$

- ▶ 当语言模型中的 n 越大，则模型越复杂，参数也越多
- ▶ 数据量足够大的情况下，模型阶数越高，历史信息越准确，句子概率的计算也越准确





模型参数

► 模型复杂度

- 设词典大小为M，试估计N-gram模型要估计的参数空间大小

参数形式	参数数量
$P(w_i)$	$ V $
$P(w_i w_{i-1})$	$ V \times V = V ^2$
$P(w_i w_{i-2}w_{i-1})$	$ V \times V \times V = V ^3$
...	...
$P(w_i w_{i-n+1} \dots w_{i-1})$	$ V \times \dots \times V = V ^n$

估计的参数数目为： $M + M^2 + \dots + M^N = (M^{(N+1)} - M)/(M - 1)$ ，假定M=1000，N=4，则需要估计约 10^{12} =1万亿个参数，参数空间巨大！



N-gram模型

▶ 马尔科夫假设

- ▶ 假设第t个词的出现跟它前面n-1个词的出现有关，即：

$$P(w_t|w_1, w_2, \dots, w_{t-1}) = P(w_t|w_{t-n+1}, \dots, w_{t-1})$$

- ▶ 这种假设下的语言模型称为n 元文法(n-gram)模型
- ▶ 实践中，一般设n=1或者2



N-gram模型

► unigram 模型(独立性假设)

- 每个词的出现都是独立的，跟其他词无关

$$P(x) = P(w_1)P(w_2)P(w_3)...P(w_n)$$

► bigram 模型(一阶马尔科夫假设)

- 每个词的出现只跟其前面一个词有关

$$P(x) = P(w_1)P(w_2|w_1)P(w_3|w_2)...P(w_n|w_{n-1})$$

► n-gram 模型

- 假设任意一个词 w_t 的出现仅与它前面的 $n-1$ 个词有关

$$P(w_t|w_{t-n+1},..., w_{t-1})$$



N-gram模型

▶ 句子加首尾标志

- ▶ 为了保证条件概率在 $i=1$ 时有意义，可以在句子首尾两端增加两个标志: $\langle s \rangle w_1, w_2, \dots, w_m \langle /s \rangle$

s' =John read a book

$s=\langle s \rangle$ John read a book $\langle /s \rangle$

- ▶ Unigram: $\langle s \rangle$, John, read, a, book, $\langle s \rangle$
- ▶ Bigram: ($\langle s \rangle$ John), (John read), (read a), (a book), (book $\langle /s \rangle$)
- ▶ Trigram: ($\langle s \rangle$ John read), (John read a), (read a book), (a book $\langle /s \rangle$)



N-gram模型

▶ 句子加首尾标志

▶ $s = \langle s \rangle$ John read a book $\langle /s \rangle$

▶ 其bi-gram概率计算:

$$p(\text{John read a book}) = p(\text{John} | \langle s \rangle) \times$$

$$p(\text{read} | \text{John}) \times p(\text{a} | \text{read}) \times$$

$$p(\text{book} | \text{a}) \times p(\langle /s \rangle | \text{book})$$

具体取值与语料中的单词分布有关



N-gram模型

▶ 应用1: 音字转换问题

▶ 输入 Pinyin=hua zhong nong ye da xue,

▶ 可能的汉字串 CString:

▶ s1=话中弄也大学?

▶ s2=化中农也大学?

▶ s3=华中农业大学?

▶

▶ 利用LM的解决思路:

▶ 计算 $P(\text{CString}|\text{Pinyin})$, 取概率最大的CString



N-gram模型

► 应用1: 音字转换问题

► 模型

$$CString = \arg \max_{CString} p(CString | Pinyin)$$

$$= \arg \max_{CString} \frac{p(Pinyin | CString) \times p(CString)}{p(Pinyin)}$$

对所有的CString,
p(Pinyin)是相同的

$$= \arg \max_{CString} p(Pinyin | CString) \times p(CString)$$

$$= \arg \max_{CString} p(CString)$$

对于上述汉字串,
拼音是确定的



N-gram模型

▶ 应用2-汉语分词问题

- ▶ text="‘武汉大学生落户政策’
- ▶ 可能的分词结果
 - ▶ seg1=‘武汉|大学生|落户|政策’
 - ▶ seg2=‘武汉大学|生|落户|政策’

$$\hat{seg} = \operatorname{argmax}_{seg} P(seg|text)$$

$$= \operatorname{argmax}_{seg} \frac{P(text|seg) \times P(seg)}{P(text)}$$

$$= \operatorname{argmax}_{seg} P(text|seg) \times P(seg)$$



N-gram模型

▶ 应用2-汉语分词问题

▶ bi-gram模型计算

- ▶ $p(\text{seg1}) = p(\text{武汉} | \langle s \rangle) \times p(\text{大学生} | \text{武汉}) \times p(\text{落户} | \text{大学生}) \times p(\text{政策} | \text{落户}) \times p(\langle /s \rangle | \text{政策})$
- ▶ $p(\text{seg2}) = p(\text{武汉} | \langle s \rangle) \times p(\text{大学} | \text{武汉}) \times p(\text{生} | \text{大学}) \times p(\langle /s \rangle | \text{政策})$



N-gram模型

► N-gram的应用

- n-gram语言模型的应用非常广泛，最早期的应用是语音识别、机器翻译等问题。
- 哈尔滨工业大学王晓龙教授最早将其应用到音字转换问题，提出了“语句级拼音输入法”，后来该技术转让给微软，也就是后来的微软拼音输入法。
 - 从windows95开始，系统就会自动安装该输入法，并在以后更高版本的windows中和Office办公软件都会集成最新的微软拼音输入法。
- n年之后，各个输入法的新秀（如搜狗和谷歌）也都采用了n-gram技术。



目录

- ▶ 语言模型概述
- ▶ N-gram语言模型
 - ▶ 模型参数
 - ▶ N-gram模型
 - ▶ 参数估计
 - ▶ 数据平滑
- ▶ 模型应用
- ▶ 语言模型评估



参数估计

▶ 两个重要基础

▶ 训练语料(training data):

- ▶ 用于建立模型，确定模型参数
- ▶ 语料越大，可以利用更多的历史，模型更准确，但是总计算量也越大
- ▶ 语料越小，则存在稀疏性问题
 - 会导致 $P(s)=0$ ，需要做平滑，即对于没出现的事件也赋予一个概率

▶ 最大似然估计(maximum likelihood Evaluation, MLE)

- ▶ 用词语的相对频率计算概率的方法



参数估计

► Bigram

► 最大似然估计(Maximum Likelihood Estimate, MLE)

$$p(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}w_i)}{\sum_{w \in V} \text{count}(w_{i-1}w)}$$

通常写成

$$p(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}w_i)}{\text{count}(w_{i-1})} = \frac{c(w_{i-1}w_i)}{c(w_{i-1})}$$

例如：P(人民|中国) = count(中国, 人民)/count(中国)

► Uni-gram

$$p(w_i) = \frac{\text{count}(w_i)}{|\text{corpus}|}$$



参数估计

▶ Bigram 的模型参数估计

▶ 给定训练语料:

“John read Moby Dick”

“Mary read a different book”

“She read a book by Cher”

▶ 根据 2 元文法求句子的概率?

▶ $p(\text{John read a book})=?$

▶ 解答

$$p(\text{John} | \langle s \rangle) = 1/3, \quad p(\text{read} | \text{John}) = 1, \quad p(\text{a} | \text{read}) = 2/3$$

$$p(\text{book} | \text{a}) = 1/2, \quad p(\langle s \rangle | \text{book}) = 1/2$$

$$p(\text{John read a book}) = 1/3 \times 1 \times 2/3 \times 1/2 \times 1/2 = 0.06$$



参数估计

语料如下

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

根据 2 元文法求句子的概率?

$p(\text{John read a book})=?$



目录

- ▶ 语言模型概述
- ▶ N-gram语言模型
 - ▶ 模型参数
 - ▶ N-gram模型
 - ▶ 参数估计
 - ▶ 参数平滑
- ▶ 模型应用
- ▶ 语言模型评估



参数平滑

▶ 参数为0

▶ 语料

“John read Moby Dick”

“Mary read a different book”

“She read a book by Cher”

▶ $p(\text{Cher read a book})$

$$= p(\text{Cher} | \langle s \rangle) \times p(\text{read} | \text{Cher}) \times p(a | \text{read}) \times \\ p(\text{book} | a) \times p(\langle /s \rangle | \text{book})$$

$$p(\text{read} | \text{Cher}) = 0/1 \quad \longrightarrow \quad p(\text{Cher read a book}) = 0$$



参数平滑

▶ 问题:

- ▶ 数据匮乏(稀疏) (Sparse Data) 引起零概率问题, 如何解决?
- ▶ 历史信息越多, 数据的稀疏性越强
 - ▶ E.g. $c(w_i(\geq c)w_{i-1}w_i)$



数据平滑(data smoothing)

- ▶ 调整最大似然估计的概率值, 使零概率增值, 使非零概率下调; 消除零概率, 改进模型的整体正确率。



参数平滑

▶ 加1法平滑(Laplace法则)

- ▶ 基本思想: 每一个词的出现的次数都加1。
- ▶ 对于unigram

$$p(w_i) = \frac{c(w_i) + 1}{\sum_{w \in V} c(w) + |V|}$$

▶ 例如,

- ▶ 对于Unigram, 设 w_1, w_2, w_3 三个词的概率分别为:
 $1/3, 0, 2/3$, 加1后情况则为?

2/6, 1/6, 3/6



参数平滑

▶ 加1法平滑(Laplace法则)

▶ 对于bi-gram

$$\begin{aligned} p(w_i|w_{i-1}) &= \frac{c(w_{i-1}w_i) + 1}{c(w_{i-1}) + |V|} \\ &= \frac{c(w_{i-1}w_i) + 1}{\sum_{w \in V} c(w_{i-1}w) + |V|} \end{aligned}$$

▶ 其中， V 为语料库对应的词典



参数平滑

▶ 加1法平滑(Laplace法则)

▶ 例如, 对于语料

▶ $\langle s \rangle$ John read Moby Dick $\langle /s \rangle$

▶ $\langle s \rangle$ Mary read a different book $\langle /s \rangle$

▶ $\langle s \rangle$ She read a book by Cher $\langle /s \rangle$

▶ 平滑前的Bigram概率

▶ $p(\text{Cher}|\langle s \rangle) = 0/3$

▶ $p(\text{read}|\text{Cher}) = 0/1$

▶ $p(a|\text{read}) = 2/3$

▶ $p(\text{book}|a) = 1/2$

▶ $p(\langle /s \rangle|\text{book}) = 1/2$



参数平滑

▶ 加1法平滑(Laplace法则)

▶ 词典: $|V|=11$

▶ 平滑: $p(w_i|w_{i-1}) = \frac{c(w_{i-1}w_i)+1}{c(w_{i-1}) + |V|}$

▶ 平滑后的2-gram模型

▶ $p(\text{Cher}|\text{<s>}) = (0+1)/(3+11) = 1/14$

▶ $p(\text{read}|\text{Cher}) = (0+1)/(1+11) = 1/12$

▶ $p(\text{a}|\text{read}) = (1+2)/(3+11) = 3/14$

▶ $p(\text{book}|\text{a}) = (1+1)/(2+11) = 2/13$

▶ $p(\text{</s>}|\text{book}) = (1+1)/(2+11) = 2/13$

▶ $p(\text{Cher read a book}) = \frac{1}{14} \times \frac{1}{12} \times \frac{3}{14} \times \frac{2}{13} \times \frac{2}{13} \approx 0.00003$



参数平滑

▶ 加 α 法平滑(Laplace法则)

$$p(w_i|w_{i-1}) = \frac{c(w_{i-1}w_i) + \alpha}{c(w_{i-1}) + |V| \times \alpha}$$

- ▶ α 是超参数
- ▶ 加 α 法平滑比加1平滑更普遍



参数平滑

▶ 回退平滑(backoff)

▶ Bigram

$$\text{▶ } P(w_i|w_{i-1})_{\text{backoff}} = \lambda_1 P(w_i|w_{i-1}) + \lambda_2 P(w_i)$$

▶ Trigram

$$\text{▶ } P(w_i|w_{i-2}w_{i-1})_{\text{backoff}} = \lambda_1 P(w_i|w_{i-2}w_{i-1}) + \lambda_2 P(w_i|w_{i-1}) + \lambda_3 P(w_i)$$

$$\text{s.t. } \sum \lambda_i = 1, \lambda_i \geq 0$$

- ▶ 上述回退平滑采用了线性插值方法
- ▶ λ_i 是超参数，用于均衡不同概率



参数平滑

▶ 平滑方法小结

- ▶ 平滑是为了解决数据稀疏性问题
- ▶ 加1 (α) 平滑
 - ▶ 对数据集中所有词统计时加1 (α)
 - ▶ 实际上是一种劫富济贫的思想
- ▶ 回退back-off
 - ▶ 用低阶n-gram概率近似高阶概率，缓解高阶概率为0的情况



目录

- ▶ 语言模型概述
- ▶ N-gram语言模型
 - ▶ 模型参数
 - ▶ N-gram模型
 - ▶ 参数估计
 - ▶ 数据平滑
- ▶ 模型应用
- ▶ 语言模型评估



模型应用

▶ 加州伯克利餐厅语音

- ▶ can you tell me about any good cantonese restaurants close by
- ▶ mid priced thai food is what i'm looking for
- ▶ tell me about chez panisse
- ▶ can you give me a listing of the kinds of food that are available
- ▶ i'm looking for a good place to eat breakfast
- ▶ when is caffe venezia open during the day

- ▶ ref: stanford LM



模型应用

► bigram计数

► 统计了其中9222个句子

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0



模型应用

$$p(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}w_i)}{\text{count}(w_{i-1})}$$

► bigram 概率

► 前面的bigram计数除以unigram计数

Unigram	I	want	to	eat	Chinese	food	lunch	spend
counts	2533	927	2417	746	158	1093	341	278

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0



模型应用

▶ 通过bigram参数估计句子概率

- ▶ 此处用<s>表示句子开始，</s>表示结束.

$$\begin{aligned} P(<s> \text{ I want english food } </s>) &= \\ P(\text{I}|<s>) \times P(\text{want}|\text{I}) \times P(\text{english}|\text{want}) \\ &\times P(\text{food}|\text{english}) \times P(</s>|\text{food}) \\ &= .000031 \end{aligned}$$

- ▶ 如果不用<s>，</s>作为句子头尾则，上式为

$$\begin{aligned} P(\text{I want english food}) &= \\ P(\text{I}) \times P(\text{want}|\text{I}) \times P(\text{english}|\text{want}) \\ &\times P(\text{food}|\text{english}) \end{aligned}$$



模型应用

► bigram计数(带加1平滑)

- 每个bigram计数直接加1

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1



模型应用

► bigram 概率 (带加1平滑)

Unigram	I	want	to	eat	Chinese	food	lunch	spend
counts	2533	927	2417	746	158	1093	341	278

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

$$p(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}w_i) + 1}{\text{count}(w_{i-1}) + |V|} \quad |V| = 1446$$



模型应用

▶ 通常对概率值bigram取log后再算

- ▶ 避免下溢出
- ▶ 加法比乘法快
- ▶ 关键是改变结果

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$



目录

- ▶ 概率建模
- ▶ 语言模型概述
- ▶ N-gram语言模型
 - ▶ 参数估计
 - ▶ 数据平滑
- ▶ 模型应用
- ▶ 语言模型评价
- ▶ 其它



目录

- ▶ 语言模型概述
- ▶ N-gram语言模型
 - ▶ 模型参数
 - ▶ N-gram模型
 - ▶ 参数估计
 - ▶ 数据平滑
- ▶ 模型应用
- ▶ 语言模型评估



语言模型评估

▶ 困惑度(perplexity, PPL)

$$\text{perplexity} = \prod_{t=1}^T \left(\frac{1}{P_{\text{LM}}(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)})} \right)^{1/T}$$

- ▶ 语言模型训练完之后，对测试集中的句子计算困惑度
- ▶ 测试集中的句子是自然流畅的人类语言
- ▶ 从公式可以看出，困惑度越低越好



语言模型评估

► 语言模型的比较

n -gram model

Increasingly
complex RNNs

Model	Perplexity
Interpolated Kneser-Ney 5-gram (Chelba et al., 2013)	67.6
RNN-1024 + MaxEnt 9-gram (Chelba et al., 2013)	51.3
RNN-2048 + BlackOut sampling (Ji et al., 2015)	68.3
Sparse Non-negative Matrix factorization (Shazeer et al., 2015)	52.9
LSTM-2048 (Jozefowicz et al., 2016)	43.7
2-layer LSTM-8192 (Jozefowicz et al., 2016)	30
Ours small (LSTM-2048)	43.9
Ours large (2-layer LSTM-2048)	39.8

越低越好

Source: <https://research.fb.com/building-an-efficient-neural-language-model-over-a-billion-words/>



Thanks